

Can better handoffs make an RLM more reliable?

A follow-up to our earlier training experiments

Last time, we taught the model a basic routine.

It learned to inspect a Python workspace and calculate an answer. Reliable use of helper model calls was still an open problem.

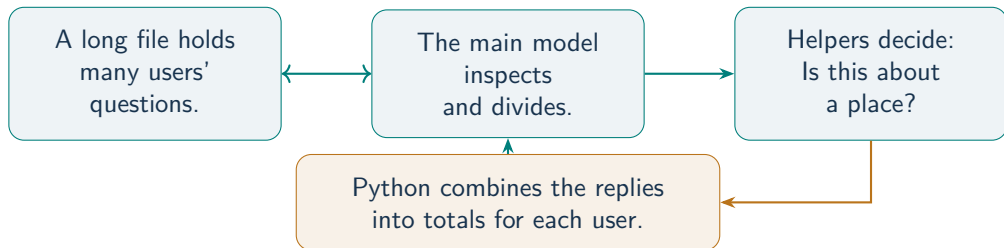
This time, we studied the handoffs.

Training improved a more demanding routine. We also found that the way helper answers are linked to their inputs can change accuracy substantially.

Alex Towell | Advisor discussion | Preliminary findings, 11 September 2026.

A long file can become many small questions whose answers Python can combine.

A recursive language model (RLM) keeps a large input in a Python workspace. The main model can inspect pieces and delegate work.

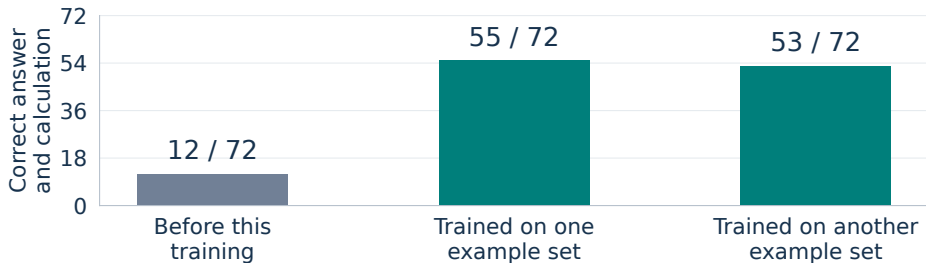


The hope: solve an unfamiliar whole by combining familiar smaller steps.

The input stays in the workspace. Helpers can divide their work again; our tests mostly use one helper layer. RLMs are existing work. Our focus is making the handoffs reliable.

Training taught a more reliable routine for asking helpers and calculating answers.

Worked examples showed how to ask a helper and calculate from its reply. We tested the trained models on newly selected records.



All models answered the same 72 questions from eight input sets. The two trained copies used different demonstrations; the helper stayed fixed. Missing outcomes allow totals of 12–19, 55, and 53–55. Familiar question types, not autonomous planning. [S3]

We changed how helper answers are linked to the text they describe.

Suppose a helper reads: *Maya bought a red bike*. It must judge two statements.

The input carries matching names.

k7ab: Maya bought a vehicle.

z2pm: The bike is blue.

Each answer carries the same name.

k7ab: Supported.

z2pm: Contradicted.

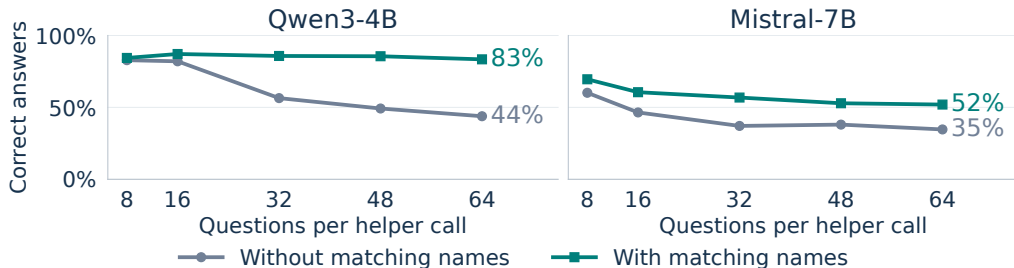
Without these names, the answers are matched only by their position in a list.

The software supplies the names. The model still has to choose the answers.

Simplified illustration. We changed the input and required output format, not the model's weights. These names are arbitrary identifiers, not hints about the correct answer.

Matching names improved accuracy when a helper answered many questions at once.

The same records were tested with and without matching names. Neither model was retrained.



Sixteen sets of records; 160 calls per model shown. Every planned answer counts. Three malformed Mistral responses count as wrong. These are helper answers, not complete RLM solutions. [H8, H9]

Smaller calls were faster than large named calls in this local test.

All methods answered the same 768 reading questions.

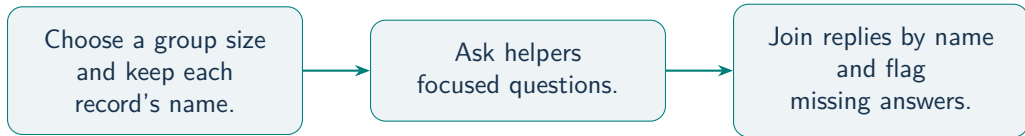
Method	Correct answers	Time
48 questions per call, no names	49%	19 seconds
48 questions per call, matching names	85%	91 seconds
16 questions per call, no names	81%	19 seconds
1 question per call, no names	87%	31 seconds

Single-question calls sent more input text. The best choice depends on which cost matters.

One Qwen3-4B test on one A100, with four simultaneous calls and no cache reuse. Times sum separate workload blocks and exclude server startup. All 848 calls returned valid answers. This is a local tradeoff, not a universal speed ranking. [E1]

Proposed RLM change: manage record links and make helper group size an explicit choice.

The surrounding program would keep track of records and answers, while the model chooses what to ask.



Test against the current RLM and smaller-call baselines, using the same models, tasks, and resource budget.

The result that matters is a more reliable **complete answer**.

Proposed component, not an established whole-RLM gain. Matching cannot guarantee correct reading, and identifiers are not a new invention. A later experiment could learn when to change group size.

We have a promising handoff result and a focused next research question.

What we know so far

Worked examples improved one useful routine. Matching inputs to helper answers improved reading accuracy without retraining.

What we want to establish next

Can a change to the RLM's handoff procedure produce more reliable complete solutions at a useful cost?

Would this be a worthwhile next study? What realistic task would make the test convincing?

The larger goal remains learning how to divide unfamiliar problems into useful smaller ones. Today's evidence is narrower. End of the main talk; the remaining pages are optional discussion material.

Example: the helper reads the text, and Python does the counting.

For each user, add the weights of their place questions. How many users have a total above 5?

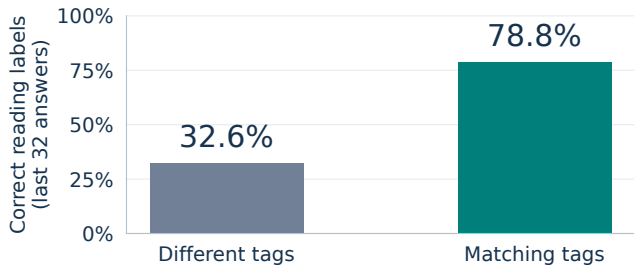
User	Question	Weight	Place question?
Ada	Where is Oslo?	4	Yes
Ada	What is the capital of Peru?	3	Yes
Ben	Who wrote Hamlet?	6	No
Ben	Where is Kyoto?	2	Yes

Ada: $4 + 3 = 7$. Ben: 2. Only Ada exceeds 5, so the answer is **1 user**.

The weights are invented test values, not confidence scores. This illustrates the task; it is not a quoted demonstration.

A control suggests that matching matters, not merely having names on the page.

Both versions gave names to inputs and answers. One reused the same names; the other used different names.



Input name → answer name

Different:

k7ab → z2pm

Matching:

k7ab → k7ab

All responses had valid formatting. Matching helped in all 16 sets of records.

Qwen3-4B; last 32 answers in 48-question calls; 3,072 labels per bar. Position specified which input to answer.

Different records from slide 5. No internal-mechanism claim. [H6]

Reward training did not improve the final-answer count in this trial.

Two runs started from the same trained model. One also rewarded agreement with a calculation based on the helper's answers.

Before reward training

57

out of 72 questions.

Correct-answer reward

55–57

out of 72 questions.

Answer reward plus the extra check

54

out of 72 questions.

Each run attempted 576 solutions. Two missing results create the range.

One exploratory pair, with 19 and 21 weight updates. These are final-answer counts, unlike slide 3's stricter calculation check. The full review of calculation correctness is not finished. [R3]

Training also helped with new combinations of familiar steps.

Example of a new combination

First find users who asked a place question. Then count those users' questions that ask for a number.

Before this training

2 / 72

correct answers and calculations.

After this training

29 / 72

correct answers and calculations.

The instructions supplied the plan. The model became better at carrying it out.

One trained model, eight previously tested input sets. Missing outcomes allow 2–13 versus 29–30 successes. These combinations were absent from our demonstrations; this is not independent planning. [S4]

Misleading names can draw an answer toward the wrong input.

Imagine answering row 7 with a box named row 8.

The model may give a sensible answer to row 8 instead of the required answer to row 7.

Names pointed to another input.

39%

of reading answers were correct.

Names pointed to no visible input.

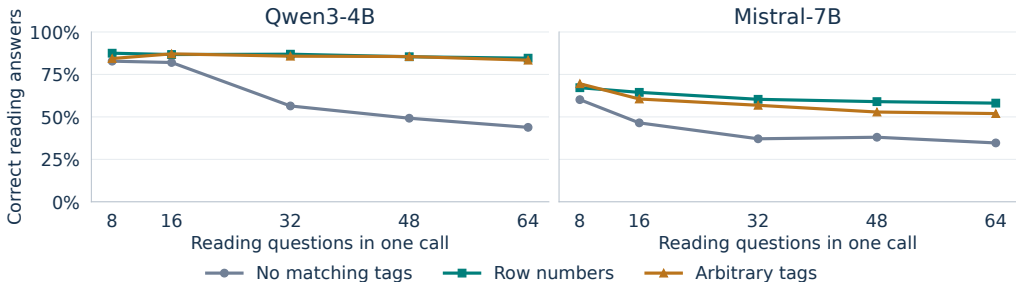
80%

of reading answers were correct.

A deliberate stress test, not normal use. Software fixed answer names and order. Sixteen batches of 48 questions; rounded percentages. Evidence about output behavior, not a measurement of attention. [H1]

Both row numbers and arbitrary names helped in the larger-batch tests.

These are the full three-format results behind slide 5.



Matching is helpful, but the Mistral model still made many errors.

Sixteen shared sets; 240 calls per model. All returned; three malformed Mistral arbitrary-name outputs count as wrong. Those failures confound comparisons between the two named formats. [H8, H9]